

Feeder-Relais (Timer-Ersatzplatine v2) — Technische Dokumentation

Projekt: 230-V-Timer-/Schaltgerät auf Basis ESP32-C3, OLED, PhotoMOS und einem **externen** Shelly 1PM Mini Gen4. Ersetzt die defekte Zeitschaltplatine eines Futterautomaten; nutzbar für jede zeitgesteuerte 230-V-Last.

Stand: 2026-08-12 · **Hardware:** Version 2 (Elektronik auf der Rückseite, Shelly extern) · **Firmware:** ESPHome, GeräteName `feeder-relais`.

Diese Datei ist die **deutsche Quelle der Wahrheit** (technische Referenz). Die **anfängertaugliche Schritt-für-Schritt-Nachbauanleitung** steht im **Handbuch** unter [docs/handbuch/](#) (HTML-Quelle + PDF, gebaut mit WeasyPrint). Übersetzungen (`DOKUMENTATION.en.md/.fr.md`) werden je Release nachgezogen.

1. Funktion und Signalkette

Drei Taster an der Front lösen je einen Timer aus, der eine 230-V-Last für eine einstellbare Zeit einschaltet:

Taste (Gehäuse)	Firmware	Standardzeit	einstellbar
S1 = Down/Manual	T1 (GPIO3)	5 s	1-600 s über Web
S2 = SET	T2 (GPIO4)	10 s	1-600 s über Web
S3 = UP	T3 (GPIO5)	15 s	1-600 s über Web

Ein 0,91"-OLED zeigt WLAN-Empfang, Uhrzeit (NTP), Status (Ruhe/Futter), Datum und freien Speicher; bei laufendem Timer den Sekunden-Countdown. Bedient wird zusätzlich über eine mobile Web-App auf Port 80 (<http://feeder-relais.local>) mit JSON-API (Kap. 6).

Signalkette:

```
Taster —> ESP32-C3 —> R1 330 Ω —> PhotoMOS K1 —> Shelly 1PM —> Last
S1-S3   (GPIO3/4/5 (GPIO6, (galv. Trennung (schaltet & (0_LAST/N
        zählen,    begrenzt  3,3 V ↔ 230 V)  misst 230 V)  an X2)
        OLED)      LED-Strom) L_F -> SW_SHELLY SW->0 intern
```

Der ESP „drückt“ für den Shelly nur den Schalter (echtes Hardware-Signal, ohne WLAN-Abhängigkeit der Kernfunktion). Der Shelly läuft in Betriebsart **Switch/Follow**: sein Relais folgt dem SW-Signal 1:1, das Timing liegt beim ESP.

2. Sicherheit

- **230 V — Lebensgefahr.** Aufbau/Prüfung/Inbetriebnahme der Netzseite nur durch Elektrofachkraft oder unterwiesene Person.
- Erste Tests (ESP, Taster, OLED, PhotoMOS-Ausgang) **ausschließlich über USB**, ohne Netzspannung.
- Erster 230-V-Test hinter **RCD/PRCD**, Gehäuse geschlossen.

- **Kriechstrecke ≥ 6 mm** zwischen jedem 230-V-Netz und Kleinspannung — als DRC-Regel hinterlegt (Kap. 3.4). Einzige Ausnahme: der PhotoMOS K1 selbst (zertifizierte Trennstelle).
- **F1 (1 A träge) + RV1 (Varistor)** primärseitig sind Pflicht.
- PhotoMOS ≥ 400 V Sperrspannung (G3VM-601AY2 o. ä.), **kein** PC817.

3. Die Platine v2

Maße: 101,6 × 77,5 mm, 4 Lagen, 1,6 mm FR4. Quelle: kicad-v2/ (Timer-Ersatzplatine-v2.kicad_pcb/.kicad_sch/.kicad_pro/.kicad_dru).

3.1 Vorder- und Rückseite

Die **gesamte Elektronik liegt auf der Rückseite** (ESP, Netzteil, PhotoMOS, 230-V-Klemmen). Auf der Vorderseite (zur Frontplatte) bleiben nur die drei Taster SW1-3 und die OLED-Buchse J2 — alles, was bedient bzw. gesehen wird. Grund: über der Platine sind nur ~16,5 mm Bauraum, nach unten (zur Front) fast nichts.

3.2 Lagenaufbau und Masseflächen

4 Kupferlagen: **F.Cu · In1.Cu · In2.Cu · B.Cu**. Die beiden Innenlagen sind **GND-Masseflächen** — aber **nur im Kleinspannungsbereich**. Im Netzspannungsbereich gibt es **kein Innenkupfer**; die 230-V-Netze verlaufen ausschließlich auf den Außenlagen F.Cu/B.Cu. Die GND-Zonen-Outlines sind geometrisch so geformt, dass sie überall ≥ 6 mm von den 230-V-Netzen wegbleiben.

3.3 Netzklassen

Aus .kicad_pro (unverändert gegenüber v1 — **nicht ändern**):

Netzkategorie	clearance	track_width	Netze
Default	0,2 mm	0,2 mm	Kleinspannung/Signal
230V	1,0 mm	1,0 mm	L_IN, L_F, N, SW_SHELLY, O_LAST

3.4 Die 6-mm-Regel (.kicad_dru)

```
(version 1)
(rule "230V->SELV 6mm (K1-Barriere ausgenommen)"
  (condition "(A.NetName=='/L_IN' || A.NetName=='/L_F' || A.NetName=='/N'
    || A.NetName=='/SW_SHELLY' || A.NetName=='/O_LAST') && !(<gleiche Netze>)
    && B.NetName != '' && B.NetName != 'Net-(K1-Pad1)'
    && !A.memberOfFootprint('K1') && !B.memberOfFootprint('K1')
    && !A.intersectsCourtyard('K1') && !B.intersectsCourtyard('K1'))"
    (constraint creepage (min 6mm))
    (constraint clearance (min 6mm)))
```

K1 ist ausgenommen, weil sein Gehäuse die zertifizierte Barriere ist (Pins 1/2 = SELV, Pins 3/4 = Netz). **Hinweis:** Beim Neufüllen der Zonen sitzt der 6-mm-Rückzug in der **Zonen-Outline** (Geometrie), nicht in der Fill-Clearance — headless ZONE_FILLER liefert danach einen sauberen DRC.

4. GPIO-Belegung — Board ↔ Firmware (verbindlich)

Dies ist der maßgebliche Abgleich zwischen **Platine** (U1-Pads → Netz, aus kicad-v2/) und **Firmware** (firmware/timer-relais-c3.yaml). Beide Seiten stimmen exakt überein.

ESP-Pin	Firmware (ID / Konfiguration)	Board: U1-Pad → Netz	Funktion
GPIO3	btn_s1 — Eingang, Pull-up, invertiert, 30 ms	Pad 3 → /BTN1	Taster S1 (Down/Manual) nach GND
GPIO4	btn_s2 — Eingang, Pull-up, invertiert, 30 ms	Pad 4 → /BTN2	Taster S2 (SET)
GPIO5	btn_s3 — Eingang, Pull-up, invertiert, 30 ms	Pad 5 → /BTN3	Taster S3 (UP)
GPIO6	shelly_trigger — GPIO-Ausgang	Pad 6 → /PMOS_DRV	über R1 → K1-LED-Anode; HIGH = Last ein
GPIO7	i2c: sda, 400 kHz	Pad 7 → /SDA	I ² C-Daten → OLED J2.4
GPIO8	status_led — WS2812 (GRB), gedimmt	modulintern, kein Board-Netz	Onboard-RGB-Status-LED (Ampel)
GPIO9	i2c: scl, 400 kHz	Pad 9 → /SCL	I ² C-Takt → OLED J2.3
5V / GND	Versorgung	/+5V / GND	vom Netzteil PS1
3V3	Onboard-LDO-Ausgang	/+3V3	→ OLED-VCC (J2.2)

Kritisch (Fix 2026-08-12): OLED-SDA muss auf **GPIO7** liegen, **nicht GPIO8** — auf GPIO8 sitzt die Onboard-WS2812. Lag SDA auf GPIO8, blieb das OLED schwarz und das LED-Signal störte die Datenleitung. Die v2-Platine wurde entsprechend korrigiert (Schaltplan-Pin 8→7, Layout, Zonen neu gefüllt; Netzliste /SDA→U1.7, /SCL→U1.9, U1.8 unbelegt; DRC sauber). Firmware unverändert (SDA=GPIO7 war stets korrekt).

Bewusst gemieden: GPIO2/8/9 als Taster (Strapping/WS2812). SCL bleibt GPIO9 (bootkompatibel mit I²C-Pullup).

5. Bauteile, Netze und Steckverbinder

5.1 Stückliste (BOM, v2)

Ref	Bauteil	Wert / Typ	Genaue Funktion
U1	ESP32-C3 Super Mini	18×24 mm, USB-C	Controller+WLAN; interner LDO 3,3 V; Onboard-WS2812 (GPIO8)
K1	PhotoMOS	Omron G3VM-601AY2 (o. -601BY / AQY216)	galv. Trennung 3,3 V↔230 V, ≥ 400 V
R1	Widerstand	330 Ω	LED-Vorwiderstand K1: (3,3–1,2 V)/330 ≈ 6 mA
PS1	AC/DC-Modul	TSP-05, 5 V/3 W (HLK-PM05-Klasse)	230 V (L_F,N) → 5 V/600 mA
C1	Elko	220 µF/10 V	Puffer +5 V (WLAN-Spitzen)
C2	Kerko	100 nF	Abblockung +5 V
F1	Feinsicherung	1 A träge, 5×20 mm	in Reihe L_IN→L_F

Ref	Bauteil	Wert / Typ	Genaue Funktion
RV1	Varistor	S14K275	Überspannungsschutz L_F N
J2	OLED	SSD1306 128×32, I²C 0x3C	aufgesteckt (GND/VCC/SCL/SDA)
SW1-3	Taster	6×6 mm, Hub 1,5 mm	T1/T2/T3 nach GND, Position fix
X1	Klemme	Netzeingang	1=N, 2=L_IN
X2	Klemme	Lastabgang	1=O_LAST, 2=N
X3	Klemme	Snubber	1=O_LAST, 2=N (optionales RC-Glied)
J1	Klemme/Stift	Shelly-Anschluss	1=SW, 2=O, 3=L, 4=N
REF1-4	Bohrung	M4 (4,3 mm)	Befestigung
—	Shelly 1PM Mini Gen4	S4SW-001P8EU	extern, max. 8 A/240 V

5.2 Netzliste (aus kicad-v2/)

/L_IN, /L_F, /N, /SW_SHELLY, /O_LAST (230 V) · /+5V, /+3V3, GND (Versorgung) · /BTN1, /BTN2, /BTN3, /PMOS_DRV, /SDA, /SCL (Signal) · Net- (K1-Pad1) (R1↔K1-LED-Anode).

Netz	führt	Weg
L_IN	230 V (ungesichert)	X1.2 → F1.2
L_F	230 V (abgesichert)	F1.1 → RV1, PS1.2, K1.3, J1.3(L)
N	230 V Neutral	X1.1 → RV1, PS1.1, J1.4(N), X2.2, X3.2
SW_SHELLY	230 V geschaltet	K1.4 → J1.1(SW)
O_LAST	230 V geschaltet	J1.2(O) → X2.1, X3.1
+5V	5 V	PS1.3 → U1.5V, C1.1, C2.1
GND	Masse	PS1.4 → U1.GND, K1.2, C1.2, C2.2, J2.1, Innenlagen
+3V3	3,3 V	U1.3V3 → J2.2 (OLED-VCC)
PMOS_DRV	Signal	U1.6 → R1.1
Net-(K1-Pad1)	Signal	R1.2 → K1.1 (LED-Anode)
SDA / SCL	I²C	U1.7/U1.9 → J2.4/J2.3
BTN1/2/3	Signal	U1.3/4/5 → SW1/2/3 → GND

5.3 Steckverbinder-Pinbelegung

Verbinder	Pin 1	Pin 2	Pin 3	Pin 4
X1 Netz	N	L_IN	—	—
X2 Last	O_LAST	N	—	—
X3 Snubber	O_LAST	N	—	—
J1 Shelly	SW_SHELLY	O_LAST	L_F	N
J2 OLED	GND	+3V3	SCL	SDA
K1 PhotoMOS	LED-Anode (→R1)	GND (LED-Kathode)	L_F	SW_SHELLY
PS1 Netzteil	N (AC)	L_F (AC)	+5V	GND

6. Firmware (ESPHome)

Datei `firmware/timer-relais-c3.yaml` (Gerätename **feeder-relais**, mDNS `feeder-relais.local`). Enthält: WLAN mit Fallback-Hotspot („Feeder-Relais Setup“, Captive-Portal), NTP-Uhr (`de.pool.ntp.org`, Europe/Berlin), mobile Web-App + JSON-API (`firmware/timer_web.h`), persistente Netzwerk-Konfig (`firmware/net_config.h`), Log-Ringpuffer (`firmware/log_ring.h`), Tasterlogik mit Entprellung/Langdruck/Menü, OLED (SSD1306 128×32, 0x3C, I²C 400 kHz) und die Onboard-WS2812 als Status-LED.

6.1 Erstes Flashen (USB)

Fertige Images liegen unter `firmware/build/` (`feeder-relais.factory.bin` für Erstflash @0x0, `feeder-relais.ota.bin` für Web-Update) — **ohne** einkompilierte WLAN-Daten. Selbst bauen: `./firmware/build_images.sh`.

- **Weg A (ESPHome):** `esphome run firmware/timer-relais-c3.yaml` (keine `secrets.yaml` nötig). Linux ggf. Benutzer in Gruppe `dialout`.
- **Weg B (Browser):** <https://espressif.github.io/esptool-js/>, Flash-Adresse 0, `feeder-relais.factory.bin`. Bei Verbindungsproblemen Boot-Modus erzwingen (BOOT halten, RESET tippen, BOOT loslassen).

6.2 WLAN einrichten

Nach dem Flashen Hotspot „**Feeder-Relais Setup**“ (Passwort `feeder1234`) → Captive-Portal (<http://192.168.4.1>) → Heim-WLAN wählen. WLAN-Daten liegen unter festem Speicher-Schlüssel und überstehen OTA-Updates. Werksreset = Factory-Image mit „Erase all flash“.

6.3 Web-Bedienung und JSON-API (Port 80)

`firmware/timer_web.h` liefert die Seite als eigener `AsyncWebHandler` auf `web_server_base` (natives ESPHome-Web-UI aus). Fünf Reiter: **Start** (T1/T2/T3 + Countdown + Stopp), **Zeiten** (Sprache DE/EN/FR + 3 Zeiten), **Netzwerk** (WLAN, IP DHCP/statisch, NTP, Hostname, Roaming, Neustart), **Status** (Version, Laufzeit, Heap, WLAN mit Signalbalken, IP/MAC, Reset-Grund, Relais), **Service** (Log, Firmware-Upload, Neustart). Kopfzeilen-Ampel = gedimmte Onboard-WS2812.

Endpoint	Wirkung
GET /api/status	JSON: active, remaining, relay, last, times[3], host, ip, ssid, rssi, mac, ap, fw, ver, uptime, heap, wifi, reset, lang, roaming
POST /api/trigger?button=N	Taster N (1-3) mit dessen Zeit auslösen
POST /api/trigger?seconds=N	ad-hoc N Sekunden schalten
POST /api/stop	sofort aus
POST /api/config?time1=A&time2=B&time3=C	Zeiten setzen (1-600 s, persistent)
GET/POST /api/net	Netzwerk-Konfig lesen/schreiben (<code>static, ip, gw, sn, dns, ntp, host, roaming, lang</code>)
POST /api/wifi?ssid=&pw=	WLAN setzen
POST /api/reconnect	WLAN neu verbinden (Roaming sofort wirksam)
POST /api/reboot	Neustart
	Log-Ringpuffer als JSON

Endpoint	Wirkung
GET /api/log? level=N&since=M	

Persistenz (wichtig): Web-Handler laufen im Webserver-Task; ESPHome-Prefs sind nicht threadsicher. `netcfg_save()` setzt nur ein Flag, das eigentliche `sync()` erledigt das 1-s-Interval im Main-Task (`net_config.h`). Statische IP per ESP-IDF-netif bei `wifi.on_connect`; Hostname zur Laufzeit (`mdns_hostname_set` + DHCP-Client-Neustart); NTP `esp_sntp_setservername`; Roaming 802.11k/v (`set_btm/set_rrm`) über `g_netcfg.roaming`.

6.4 Bedienung an Tasten und OLED

- **kurz S1/S2/S3** → Timer 1/2/3 mit eingestellter Zeit.
- **lang UP (S3) ≥ 1,2 s** → alle Timer stoppen, aus.
- **lang SET (S2) ≥ 3 s** → Info-Menü (Ansicht: WLAN, IP, Zeit, System; S1/S3 blättern, kurz SET schließt).

OLED: oben WLAN-Balken · Uhr bzw. Countdown · Status (Ruhe/Futter) / Menüseite; unten Datum · freier Speicher. Wochentag/Status/Menütitel folgen der Sprache.

6.5 Updates (OTA)

- **Netzwerk-OTA:** `esphome run ...` (Port 3232, `ota: platform: esphome`).
- **Web-Upload:** Reiter Service → `.bin` hochladen (`ota: platform: web_server`, POST /update auf Port 80). Gerät startet danach neu.

7. KiCad und Fertigung (v2)

Projekt in `kicad-v2/` öffnen (getestet KiCad 8/9/10). Zum reinen Nachbauen nicht nötig — Fertigungsdaten liegen bei.

1. **Testdruck:** `box/Timer-Ersatzplatine-v2-BOARD.stl` 1:1 drucken und im Gehäuse-Rückteil anprobieren (Umriss, Dom-Schlitze, M3-Löcher).
2. **Gerber + Bohrdaten:** *Datei* → *Fertigungsdaten*; alle Standardlagen + Excellon; als ZIP.
3. **Bestellung: 4 Lagen**, 1,6 mm FR4, HASL/ENIG, 1 oz Cu. (Fertige Gerber liegen als `kicad-v2/Timer-Ersatz-Gerber/` bzw. `.zip` bei.)
4. **Bei eigenen Änderungen:** Netzklassen (Kap. 3.3) und die 6-mm-Regel (Kap. 3.4) nicht aufweichen; nach Änderungen DRC fehlerfrei bekommen und die GND-Zonen neu füllen (der 6-mm-Rückzug steckt in der Outline).

Schaltplan-Altlast: Der v2-Schaltplan enthält noch Symbole **J3-J6**, **SW4**, **X4** (OLED-Alternativen + entfernter Shelly/Snubber), die nicht auf dem PCB sind. Harmlos für die Fertigung, aber „Update PCB from Schematic“ würde sie einfügen wollen → vor Release aus dem Schaltplan entfernen.

8. Gehäuse (`box/`)

Zweiteilig: **Vorderteil** (Original-Frontplatte mit Sichtfenster + Tast-Stößeln) und **Rückteil** (3D-Druck, `box/feeder_back.scad` → `feeder_back_35mm.stl`).

Maß	Wert
Rückteil außen	109,8 × 90,8 mm
Tiefe (v2)	35 mm (Original war 5,7 mm)
Wand-/Bodenstärke	1,3 mm
Schraubdome	6 Stück, Raster 45 mm (X) × 70 mm (Y), Ø 8 mm, Loch 3,2 mm, Senkung 6/2 hinten
Aufhängung	Lasche mit Schlüsselloch (Wandmontage)
Platine	101,6 × 77,5 mm

Die Platine wird von hinten an die sechs Dome geschraubt; alle hohen Bauteile sitzen auf der Rückseite (Höhenbudget zur Front ist knapp).

9. Montage und Inbetriebnahme

1. Kleinteile bestücken (flach → hoch): R1, C2, K1, Taster (exakt!), J2-Buchse, C1, F1-Halter, RV1, Klemmen.
2. ESP per USB flashen + WLAN testen (Kap. 6), dann einlöten.
3. **Nur-USB-Test** (kein Netz): Taster starten Countdown, OLED zeigt an, PhotoMOS-Ausgang schaltet (Multimeter an K1-Ausgang bzw. SW_SHELLY).
4. PS1 einlöten, Shelly an J1 (SW/O/L/N), Last an X2, ggf. Snubber an X3.
5. Sicht-/Durchgangsprüfung: kein L/N-Schluss, ≥ 6 mm Abstände, keine Lötspitzer.
6. **Erster Netztest:** Gehäuse zu, über PRCD/RCD; OLED zeigt Uhr → Shelly einrichten (WLAN, Input Mode **Switch/Follow**) → Tastertest mit Last.
7. Zeiten über <http://feeder-relais.local> einstellen.

10. Dateiübersicht

Pfad	Inhalt	Regenerierbar?
kicad-v2/ *.kicad_pcb/.sch/.pro/.dru	Platine v2 (Layout, Schaltplan, Projekt, DRC-Regel)	Hand-gepflegt
kicad-v2/fp-lib-table	Footprint-Bibliothekstabelle	Hand-gepflegt
kicad-v2/Timer-Ersatz-Gerber(.zip)	Fertigungsdaten	ja (aus KiCad)
box/Timer-Ersatzplatine-v2-BOARD.stl	Platinen-Attrappe (Anprobe)	ja (kicad-cli)
box/feeder_back.scad	Gehäuse-Rückteil (OpenSCAD)	Quelle
box/feeder_back_35mm.stl	druckfertiges Rückteil	ja (OpenSCAD)
firmware/timer-relais-c3.yaml	ESPHome-Konfiguration	Hand-gepflegt
firmware/timer_web.h	Web-App + JSON-API	Hand-gepflegt
firmware/net_config.h	persistente Netzwerk-Konfig	Hand-gepflegt
firmware/log_ring.h	Log-/Debug-Ringpuffer	Hand-gepflegt
firmware/build/*.bin	Flash-Images (factory + ota)	generiert
docs/DOKUMENTATION.md	diese Referenz (deutsche Quelle)	Hand-gepflegt
docs/handbuch/	Handbuch (HTML-Quelle + PDF, WeasyPrint)	teils generiert (PDF)
docs/PROJEKTPLAN.md	Phasen/Status (nur Deutsch)	Hand-gepflegt

Pfad	Inhalt	Regenerierbar?
tools/md2pdf.py	Markdown→PDF für diese Doku	Quelle

11. Änderungshistorie

Version	Änderung
v1	Erstentwurf: Shelly + Snubber auf der Platine, zweilagig, Elektronik teils vorne.
v2	Elektronik komplett auf die Rückseite ; Shelly extern (Anschluss J1), Snubber extern (X3); 4-lagig mit GND-Masseflächen (nur Kleinspannung); Netzspannung nur auf F/B; 6-mm-Kriechstrecke als DRC-Regel (K1 ausgenommen); Außenmaß 101,6 × 77,5 mm; Gehäuse-Rückteil box/ (3D-Druck, 35 mm tief); Steckverbinder umbenannt (X1 Netz, X2 Last, X3 Snubber, J1 Shelly, J2 OLED).
v2-Fix 2026-08-12	OLED-SDA von GPIO8 auf GPIO7 korrigiert (GPIO8 = Onboard-WS2812). Schaltplan-Pin 8→7 + Layout + Zonen; verifiziert (Netzliste, DRC). Firmware unverändert.